Laboratorio 5. Un canal HTTP a la vista

Grupo 1 | Guía para reproducir la parte insegura | 8 de noviembre de 2026

Guía en PDF · Descargar el ejemplo y la guía en ZIP

1. Qué vamos a comprobar

Un cliente y un servidor intercambian datos a través de un canal de comunicación. En este ejercicio el cliente será curl y el servidor será un programa pequeño de Node.js. Ambos se ejecutan en el mismo computador. Curl enviará un mensaje por HTTP y el servidor responderá que lo recibió.

La petición funcionará. Pero funcionará sin TLS: no habrá cifrado del canal ni un certificado que permita comprobar la identidad del servidor. Con Wireshark buscaremos el mensaje dentro del tráfico capturado. Leerlo desde allí será la evidencia de que el contenido viajó en texto plano.

Usamos un ejemplo independiente para que cualquier grupo pueda repetirlo. No hace falta instalar Vault TV, Docker, Python ni una base de datos. Tampoco hay un inicio de sesión: `usuario` es un dato ficticio del mensaje, no una cuenta que el servidor autentique.

Herramientas y archivos

Necesitas Node.js 22 o posterior, curl y Wireshark. En Windows, Wireshark debe contar con Npcap para capturar loopback. Si no aparece ese adaptador, revisa su instalación antes de seguir.

Descarga el ZIP, usa **Extraer todo** y abre la carpeta extraída. En GitHub puede ser necesario pulsar **Download raw file** para descargarlo. Si no tienes acceso al repositorio, pide a quien comparte la guía que te envíe el ZIP: una vez descargado, funciona por su cuenta. El paquete contiene esta guía, el PDF y una carpeta `ejemplo-http` con tres archivos: `server.mjs`, `handler.mjs` y `solicitud.json`. Puedes guardarlo donde quieras.

En esa carpeta abre PowerShell. Todos los comandos de Windows se ejecutan desde la carpeta que contiene `ejemplo-http`, no desde dentro del ZIP. Comprueba:

```
node --version
curl.exe --version
```

`server.mjs` abre el puerto HTTP. `handler.mjs` lee el JSON y prepara la respuesta; después podrá reutilizarse con un servidor HTTPS. `solicitud.json` contiene únicamente estos datos de prueba:

```
{
  "usuario": "estudiante-demo",
  "mensaje": "LAB_CANAL_INSEGURO: mensaje de prueba para el laboratorio."
}
```

Si ya tienes el repositorio, estos mismos archivos están en `labs/secure-channel`. Utiliza la rama `feature/lab-05-secure-channel-g1`. El ensayo anterior con Core se conserva como antecedente y tiene un procedimiento distinto.

2. Arrancar el servidor y preparar Wireshark

Terminal 1: servidor

Desde la carpeta extraída ejecuta:

```
node ejemplo-http/server.mjs
```

Debe aparecer:

```
Servidor del laboratorio: http://127.0.0.1:18080/mensaje
Déjalo abierto. Para terminar, pulsa Ctrl+C en esta terminal.
```

Conserva esa terminal abierta. Node atiende las peticiones mientras el programa está en ejecución. La dirección `127.0.0.1` es loopback: el tráfico vuelve al mismo computador y no sale por Wi-Fi. Por eso capturaremos en el adaptador de loopback.

Wireshark: antes de enviar el mensaje

1. Abre Wireshark. Entra en **Capture** → **Options** o **Captura** → **Opciones**. También puedes usar **Ctrl+K**.
2. Selecciona solamente **Adapter for loopback traffic capture**. Comprueba que no queden otras interfaces seleccionadas.
3. En **Capture filter for selected interfaces / Filtro de captura para interfaces seleccionadas** escribe:

```
tcp port 18080
```

4. Toma la primera captura de pantalla. Deben verse el adaptador elegido y el filtro. Luego pulsa **Start / Iniciar**.

Este es un filtro de captura: decide qué paquetes se guardan. Más adelante usaremos otro filtro para elegir cuáles se muestran. No tienen la misma sintaxis.

> **Captura 1 pendiente.** Aquí irá la imagen real de las opciones de Wireshark: adaptador de loopback seleccionado y filtro `tcp port 18080`.

No envíes todavía la petición si Wireshark no está capturando. Una captura iniciada después no recupera los paquetes que ya pasaron.

3. Enviar el mensaje y guardar la captura

Terminal 2: cliente

Abre otra ventana de PowerShell en la misma carpeta que contiene `ejemplo-http`. Deja el servidor funcionando en la primera. Cuando Wireshark esté capturando, ejecuta:

```
curl.exe --noproxy "*" --http1.1 --include `
  --header "Content-Type: application/json" `
  --data-binary "@ejemplo-http/solicitud.json" `
  http://127.0.0.1:18080/mensaje
```

El acento grave al final de cada una de las tres primeras líneas permite continuar el comando en PowerShell. No añadas espacios después de él. `--data-binary` envía el archivo y hace que curl use POST. `--include` muestra también las cabeceras de la respuesta. `--noproxy` mantiene la petición local aunque el equipo tenga un proxy configurado.

En la terminal deben aparecer `HTTP/1.1 200 OK` y un JSON con `recibido: true`, el usuario de prueba y el mensaje completo. El estado 200 confirma que el servidor atendió la petición; por sí solo no demuestra que la conexión estuviera protegida.

Volver a Wireshark

1. Comprueba que aparecieron paquetes y pulsa el botón rojo **Stop / Detener**. También puedes usar **Ctrl+E**.
2. Guarda la captura con **File → Save As / Archivo → Guardar como**, con el nombre `http-grupo-1.pcapng`, en la carpeta que prefieras.
3. En la barra de filtro de visualización, situada sobre la lista de paquetes, escribe y pulsa Enter:

```
tcp.port == 18080 && http
```

4. Busca una petición `POST /mensaje` y su respuesta `HTTP/1.1 200 OK`. Toma la segunda captura de pantalla con el filtro y ambas filas visibles. Los números de paquete y los puertos del cliente pueden cambiar entre ejecuciones.

> **Captura 2 pendiente.** Aquí irá la lista real de paquetes con la petición POST y la respuesta HTTP 200.

Si ves TCP pero no aparece HTTP, consulta **Decode As** en la sección de problemas frecuentes. No hace falta repetir una captura que ya contiene la conversación completa.

4. Leer la conversación

La lista de paquetes muestra que hubo un intercambio. Ahora vamos a leer su contenido.

1. Selecciona la fila `POST /mensaje`.
2. Pulsa el botón derecho y elige **Follow → TCP Stream / Seguir → Flujo TCP**. También está en **Analyze / Analizar**.
3. Elige **Entire conversation / Conversación completa** y **Show data as: ASCII**. Según el idioma de Wireshark, los nombres pueden variar un poco.
4. Localiza `POST /mensaje HTTP/1.1`, el usuario `estudiante-demo` y el marcador `LAB_CANAL_INSEGURO` dentro del JSON enviado.
5. Busca también `HTTP/1.1 200 OK` y el JSON de respuesta. Debe contener el mismo mensaje y `"recibido":true`.
6. Toma la tercera captura de pantalla. Ajusta el tamaño de la ventana para que se lean la solicitud y la respuesta, manteniendo visible el título de Wireshark o de Follow TCP Stream.

> **Captura 3 pendiente.** Aquí irá la conversación real reconstruida en Follow TCP Stream, con el JSON de la petición y la confirmación del servidor legibles.

Wireshark reúne los bytes de esa conexión TCP para mostrarlos seguidos. No está consultando una base de datos ni leyendo el archivo original: trabaja con los paquetes que acabas de capturar. Puedes cerrar el servidor y volver a abrir el archivo `.pcapng`; la conversación seguirá disponible en la captura.

Cuando termines, vuelve a la terminal del servidor y pulsa **Ctrl+C**. Puedes cerrar la otra terminal. Para repetir el ejercicio, arranca el servidor, inicia una captura nueva y vuelve a enviar el mismo archivo.

Qué enviar para completar el informe

Guarda las tres imágenes como PNG. Evita incluir otras ventanas o información personal. Si la conversación no cabe en una imagen, envía dos capturas consecutivas de esa misma ventana. Conserva también el PCAP en tu computador para revisar cualquier duda.

Junto con las imágenes, anota la fecha, las versiones de Node.js, curl y Wireshark, si recibiste 200 y si pudiste leer el marcador en Follow TCP Stream. El PDF se completará con esas imágenes y con lo que realmente observaste.

5. Qué demuestra el ensayo

Si el marcador se lee en Follow TCP Stream, el cuerpo de la petición quedó expuesto a quien pudo capturar ese tráfico. En este ejemplo también se ve la respuesta. JSON organiza el contenido, pero no lo cifra. TCP se encarga del transporte; tampoco aporta cifrado por sí mismo.

El problema no depende de Node.js o curl. Lo provoca el uso de HTTP sin TLS en este trayecto. En una aplicación con información sensible, esa exposición podría permitir leer datos que solo deberían conocer el cliente y el servidor. Aquí usamos un mensaje ficticio para comprobarlo sin introducir contraseñas ni cuentas reales.

Hay un límite importante: el observador necesita acceso al trayecto del tráfico. En esta práctica capturamos loopback desde el mismo equipo. El resultado no demuestra que cualquier persona conectada al mismo Wi-Fi pueda leer todas las conversaciones de los demás. Tampoco prueba modificación de mensajes, suplantación o una intrusión.

Estado de esta edición

El 8 de noviembre de 2026 se ejecutó el servidor en Windows con Node.js **24.14.1** y curl **8.13.0**. La solicitud válida recibió 200 y devolvió el marcador completo. También se comprobaron las respuestas a JSON inválido (400), método incorrecto (405), ruta inexistente (404) y tipo de contenido incorrecto (415). Wireshark **4.4.7** reconoce el adaptador de loopback instalado.

La captura manual y su inspección visual están pendientes. Las tres posiciones reservadas en esta guía todavía no son evidencia. La captura anterior del ensayo con Core corresponde a otro procedimiento y no se utiliza como resultado de este ejemplo.

Comprobación manual	Resultado que debes registrar
Interfaz y filtro	Solo loopback; filtro `tcp port 18080`.
Petición y respuesta	POST `/mensaje` y respuesta 200 presentes.
Lectura del cuerpo	Marcador y usuario legibles en Follow TCP Stream.
Evidencia	Fecha, versiones y tres imágenes reales.

Continuidad para los grupos 2 y 3

Conserven `POST /mensaje`, el archivo `solicitud.json` y la respuesta del ejemplo. Así podrán comparar el mismo intercambio antes y después de protegerlo.

El Grupo 2 ya publicó una receta general de TLS. Su aplicación en Vault TV usa Core y el puerto 18081; este ejemplo independiente usa Node.js y el puerto 18080.

Para proteger este ejemplo puede reutilizar `handleRequest`, exportado por `handler.mjs`, al crear su servidor HTTPS. Deberá generar las llaves y el certificado con OpenSSL, configurar TLS y explicar cómo el cliente confía en la CA y comprueba el nombre del servidor. Si conservan `127.0.0.1`, deberán contemplar esa IP en el SAN del certificado.

El Grupo 3 repetirá la captura con el puerto HTTPS acordado. Deberá comprobar que la petición sigue funcionando con validación del certificado y que el cuerpo no se lee en la captura normal de TLS, sin claves de descifrado. También revisará el rechazo de un certificado que no sea confiable o no corresponda al servidor. Estas comprobaciones quedan fuera del resultado del Grupo 1.

6. Si algo no sale y variante Linux

Problemas frecuentes

Lo que ocurre	Qué revisar
`node` no se reconoce	Instala Node.js y abre una terminal nueva.
`EADDRINUSE` al arrancar	Ya hay un proceso en 18080. Si es tu servidor de esta práctica, reutilízalo o detenlo con Ctrl+C. No cierres procesos ajenos.
Curl no encuentra `solicitud.json`	Abre PowerShell en la carpeta que contiene `ejemplo-http`; comprueba que extrajiste el ZIP.
Curl no conecta	Mantén abierta la terminal del servidor y revisa la dirección y el puerto.
No aparece loopback	Revisa que Wireshark tenga Npcap instalado. En Linux se llama `lo`.
No aparecen paquetes	Selecciona loopback, inicia la captura y luego envía otra petición.
Hay TCP, pero el filtro HTTP deja la lista vacía	Borra `&& http`. Selecciona un paquete del puerto 18080, abre **Analyze** → **Decode As**, elige HTTP para ese puerto y acepta. Reaplica el filtro.
Follow TCP Stream muestra otra conversación	Cierra el diálogo y selecciona el POST de esta prueba. Revisa que el destino sea 18080.

Ubuntu u otro Linux

Esta variante indica los cambios respecto a Windows. La ejecución registrada en esta edición fue en Windows; no acredita una reproducción independiente en Linux ni WSL.

Ejecuta Node.js, curl y Wireshark en el mismo entorno Linux. En una terminal, desde la carpeta extraída:

```
node ejemplo-http/server.mjs
```

En Wireshark selecciona **lo** y usa el filtro de captura `tcp port 18080`. Inicia la captura. En otra terminal, desde la misma carpeta:

```
curl --noproxy '*' --http1.1 --include \
  --header 'Content-Type: application/json' \
  --data-binary '@ejemplo-http/solicitud.json' \
  http://127.0.0.1:18080/mensaje
```

Después detén y guarda la captura; usa los mismos filtros de visualización y Follow TCP Stream. Tu instalación debe permitir capturar en `lo`. Evita mezclar un servidor en WSL con Wireshark en Windows para esta variante: el trayecto cambia y ya no es el procedimiento que acabamos de describir.

Referencias para consultar

Captura de loopback en Wireshark, opciones de captura y Follow TCP Stream.

Servidor HTTP de Node.js y opciones de curl.